

Kubernetes Multiservice Lab

Flask API + Redis on a Real k3s Cluster

Kustomize · Secrets · Inter-service DNS · GitHub Actions CI/CD

Author	Anouar Mohamed
Platform	GitHub Codespaces (4-core, free via Student Pack)
Distribution	k3s v1.35+ (CNCF-certified Kubernetes)
Date	June 13, 2026
Cost	\$0 (GitHub Student Pack)
Repository	github.com/AnouarMohamed/k8s-multiservice-lab

Skills: Multi-service deployment · Custom container images · Kustomize base/overlay ·
Kubernetes Secrets · Inter-service DNS · ConfigMap injection · GitHub Actions CI/CD ·
YAML linting · Makefile automation

Contents

1	Overview	2
2	The Application	2
2.1	Flask API	2
2.2	Container Image	3
3	Kustomize Structure	4
3.1	Base Manifests	4
3.2	Staging Overlay	4
3.3	Previewing the Merged Output	4
4	Secrets Management	5
5	Deployment	5
5.1	Applying the Overlay	6
5.2	Accessing the Application	6
6	Inter-Service DNS Verification	6
7	GitHub Actions CI Pipeline	7
8	Makefile Automation	8
9	Repository Structure	8
10	Summary	9

1 Overview

This lab deploys a real multi-service application on a k3s Kubernetes cluster running inside GitHub Codespaces. The platform consists of a Python Flask API backed by a Redis instance, communicating over Kubernetes internal DNS. The entire platform is managed with Kustomize using a proper base/overlay pattern, Secrets handled securely and excluded from version control, and a GitHub Actions CI pipeline validating every push.

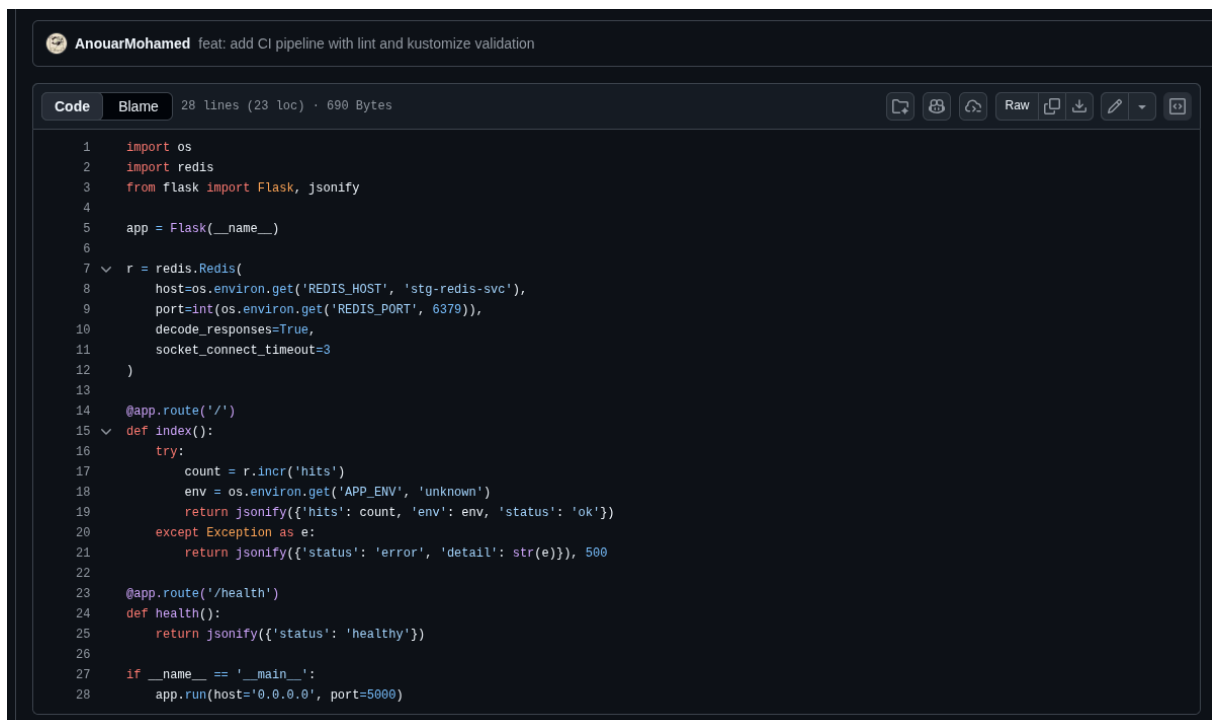
Architecture

Component	Description
Flask API	Python 3.12 + Gunicorn, 2 replicas, increments a Redis hit counter on every request
Redis	Redis 7 Alpine, 1 replica, stores the hit counter as a persistent key
Service	ClusterIP services for both — stable internal DNS names
ConfigMap	REDIS_HOST and REDIS_PORT injected as environment variables
Secret	APP_ENV and SECRET_KEY — created imperatively, never committed
Kustomize	Base manifests + staging overlay with namePrefix and label injection
CI	GitHub Actions: yamllint + kustomize build validation on every push

2 The Application

2.1 Flask API

The Flask application connects to Redis using the host injected from the ConfigMap, increments a hit counter on every request, and exposes a `/health` endpoint for Kubernetes probes:



```

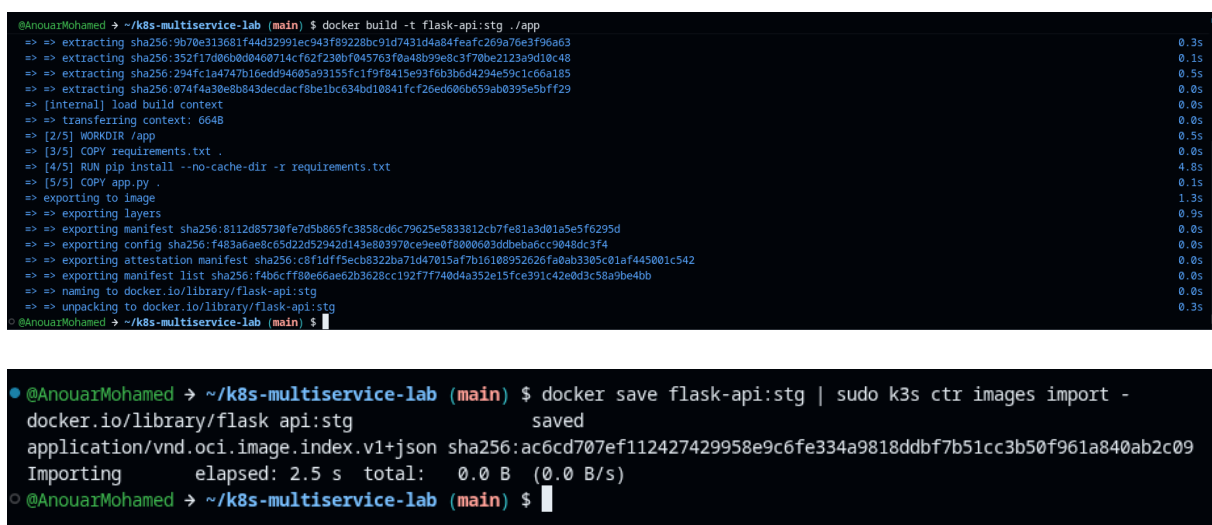
1  import os
2  import redis
3  from flask import Flask, jsonify
4
5  app = Flask(__name__)
6
7  r = redis.Redis(
8      host=os.environ.get('REDIS_HOST', 'stg-redis-svc'),
9      port=int(os.environ.get('REDIS_PORT', 6379)),
10     decode_responses=True,
11     socket_connect_timeout=3
12 )
13
14 @app.route('/')
15 def index():
16     try:
17         count = r.incr('hits')
18         env = os.environ.get('APP_ENV', 'unknown')
19         return jsonify({'hits': count, 'env': env, 'status': 'ok'})
20     except Exception as e:
21         return jsonify({'status': 'error', 'detail': str(e)}), 500
22
23 @app.route('/health')
24 def health():
25     return jsonify({'status': 'healthy'})
26
27 if __name__ == '__main__':
28     app.run(host='0.0.0.0', port=5000)

```

Figure 1: Flask application source showing Redis connection and hit counter endpoint

2.2 Container Image

The image is built locally and loaded directly into the k3s containerd runtime without a registry:



```

@AnouarMohamed ~ /k8s-multiservice-lab (main) $ docker build -t flask-api:stg ./app
=> extracting sha256:9b70e313681f44d32991ec943f89228bc91d7431d4a84feaf269a76e3f96a63 0.3s
=> extracting sha256:352f17d06b0d0460714cf62f230bf045763f0a48b99e8c3f70be2123a9d10c48 0.1s
=> extracting sha256:294fcl4747b16edd94605a93155fc1f9f8415e93f6b3b6d4294e59c1c66a185 0.5s
=> extracting sha256:074f4a30e8b843decddacfb8e1bc634bd10841fcf26ed060b659ab0395e5bfff29 0.0s
=> [internal] load build context 0.0s
=> transferring context: 664B 0.0s
=> [2/5] WORKDIR /app 0.5s
=> [3/5] COPY requirements.txt . 0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 4.8s
=> [5/5] COPY app.py . 0.1s
=> exporting to image 1.3s
=> exporting layers 0.9s
=> exporting manifest sha256:8112d85730fe7d5b865fc3858cd6c79625e5833812cb7fe81a3d01a5e5f6229d 0.0s
=> exporting config sha256:f483a6ae8c85d2d52942d143e803970ce9ee0f0800603ddebabcc9848dc3f4 0.0s
=> exporting attestation manifest sha256:c8f1dfff5ecb8322ba71d47015af7b16108952626fabab3305c01af445001c542 0.0s
=> exporting manifest list sha256:f4b6cfff08e66ae62b3628cc192f7740d4a352e15fce391c42e0d3c58a9be4bb 0.0s
=> naming to docker.io/library/flask-api:stg 0.0s
=> unpacking to docker.io/library/flask-api:stg 0.3s
@AnouarMohamed ~ /k8s-multiservice-lab (main) $

@AnouarMohamed ~ /k8s-multiservice-lab (main) $ docker save flask-api:stg | sudo k3s ctr images import -
docker.io/library/flask api:stg saved
application/vnd.oci.image.index.v1+json sha256:ac6cd707ef112427429958e9c6fe334a9818ddbf7b51cc3b50f961a840ab2c09
Importing elapsed: 2.5 s total: 0.0 B (0.0 B/s)
@AnouarMohamed ~ /k8s-multiservice-lab (main) $

```

Figure 2: flask-api:stg image listed in k3s containerd runtime after import

3 Kustomize Structure

3.1 Base Manifests

The base layer contains environment-agnostic manifests: Namespace, ConfigMap, two Deployments (Flask API and Redis), and two Services. No environment-specific values appear in the base.

```
# k8s/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- namespace.yaml
- redis-deployment.yaml
- redis-service.yaml
- api-deployment.yaml
- api-service.yaml
- configmap.yaml
```

3.2 Staging Overlay

The overlay applies three transformations to the base: it sets the namespace to **stg**, prepends **stg-** to every resource name, and injects **env: staging** as a label on every resource. Patches override the replica count and resource limits for staging:

```
# k8s/overlays/staging/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: stg
namePrefix: stg-
labels:
- pairs:
    env: staging
    includeSelectors: false
resources:
- ../../base
patches:
- path: replica-patch.yaml
  target:
    kind: Deployment
    name: flask-api
- path: resource-patch.yaml
  target:
    kind: Deployment
    name: flask-api
```

3.3 Previewing the Merged Output

`kubectl kustomize` previews the complete merged manifest without applying anything. This confirms the `namePrefix`, `namespace`, and `labels` are applied correctly before any

resources are created:

```

@AnouarMohamed → ~/k8s-multiservice-lab (main) $ kubectl kustomize k8s/overlays/staging
# Warning: 'commonLabels' is deprecated. Please use 'labels' instead. Run 'kustomize edit fix' to update your Kustomization automatically.
apiVersion: v1
kind: Namespace
metadata:
  labels:
    env: staging
  name: stg
---
apiVersion: v1
data:
  REDIS_HOST: redis-svc
  REDIS_PORT: "6379"
kind: ConfigMap
metadata:
  labels:
    env: staging
  name: stg-api-config
  namespace: stg
---
apiVersion: v1
kind: Service
metadata:
  labels:
    env: staging
  name: stg-flask-api-svc
  namespace: stg
spec:
  ports:
    - port: 80
      targetPort: 5000
  selector:
    app: flask-api
    env: staging
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:
  labels:
    env: staging
  name: stg-redis-svc

```

Figure 3: *Kustomize merged output showing stg-redis-svc with env: staging label and namespace: stg*

4 Secrets Management

Secrets are never stored in plain YAML and never committed to the repository. They are created imperatively with `kubectl create secret` using `-dry-run=client` to generate the manifest locally, applied once, and the file is gitignored:

```

@AnouarMohamed → ~/k8s-multiservice-lab/k8s (main) $ kubectl create namespace stg
cd ~/k8s-multiservice-lab/k8s/base
kubectl apply -f secret.yaml -n stg
kubectl get secrets -n stg
namespace/stg created
secret/api-secrets created
NAME          TYPE      DATA   AGE
api-secrets   Opaque    2       0s

```

Figure 4: *api-secrets created in stg namespace — Opaque type with 2 keys*

Why secrets are not committed

`kubectl create secret` encodes values in base64, which is encoding not encryption — anyone with repo access can decode them with a single command. The correct pattern for this lab: generate locally, apply once, gitignore. Production solution: Sealed Secrets, External Secrets Operator, or Vault.

5 Deployment

5.1 Applying the Overlay

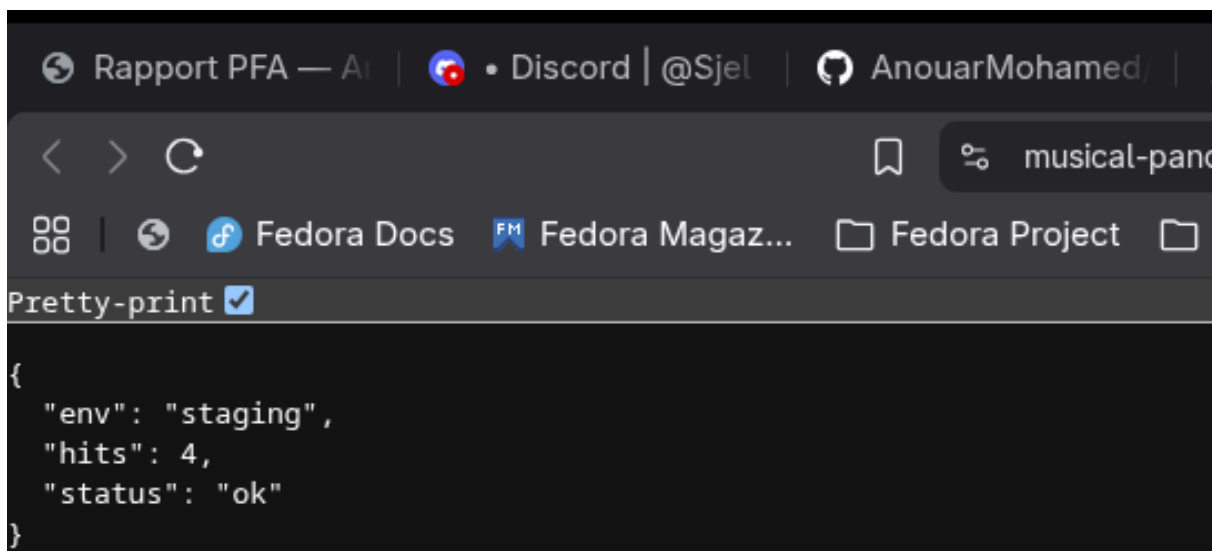
The full platform is deployed with a single command. Kubernetes schedules all three pods and the Deployment controller ensures they reach the desired state:

```
@AnouarMohamed ~ /k8s-multiservice-lab (main) $ kubectl get pods -n stg --watch
NAME                                READY   STATUS             RESTARTS   AGE
stg-flask-api-5f9c688646-n56gk      0/1     Running            0           23s
stg-flask-api-5f9c688646-rfx5m      0/1     ContainerCreating  0           23s
stg-redis-6b54f46b4c-gnjb5         0/1     ContainerCreating  0           23s
stg-flask-api-5f9c688646-rfx5m      0/1     Running            0           24s
stg-redis-6b54f46b4c-gnjb5         0/1     Running            0           25s
stg-flask-api-5f9c688646-n56gk      1/1     Running            0           30s
stg-flask-api-5f9c688646-rfx5m      1/1     Running            0           36s
stg-redis-6b54f46b4c-gnjb5         1/1     Running            0           37s
```

Figure 5: All 3 pods reach Running and READY=1/1 — 2 Flask API replicas and 1 Redis

5.2 Accessing the Application

Port-forward creates a tunnel from localhost to the Flask service:



```
{
  "env": "staging",
  "hits": 4,
  "status": "ok"
}
```

Figure 6: Hit counter incrementing on each request — confirms Flask is writing to Redis successfully

What the hit counter proves

Each curl request hits the Flask API through the Kubernetes Service. Flask calls `r.incr('hits')` on the Redis instance over the `stg-redis-svc` DNS name. An incrementing counter confirms the full communication chain: browser → port-forward → Service → Flask pod → Redis Service → Redis pod.

6 Inter-Service DNS Verification

Kubernetes assigns every Service a stable DNS name following the pattern `<service>.<namespace>.svc`. This is verified by execing into a running Flask pod and resolving the Redis service name:

```

@AnouarMohamed ~ /k8s-multiservice-lab (main) $ POD=$(kubectl get pods -n stg -l app=flask-api -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it $POD -n stg -- /bin/sh
/app # nslookup redis-svc.stg.svc.cluster.local
Server:      10.43.0.10
Address:     10.43.0.10:53

/app # nslookup stg-redis-svc.stg.svc.cluster.local
Server:      10.43.0.10
Address:     10.43.0.10:53

Name:   stg-redis-svc.stg.svc.cluster.local
Address: 10.43.144.175

/app # nc -zv stg-redis-svc 6379
stg-redis-svc (10.43.144.175:6379) open
/app #

```

Figure 7: DNS resolution of `stg-redis-svc.stg.svc.cluster.local` from inside the Flask pod

7 GitHub Actions CI Pipeline

A CI pipeline runs on every push to main and every pull request. It lints all YAML files and validates the Kustomize build without needing a live cluster:

```

name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install yamllint
        run: pip install yamllint
      - name: Lint YAML
        run: yamllint k8s/
      - name: Install kubectl
        uses: azure/setup-kubectl@v3
      - name: Install kustomize
        run: |
          curl -s "https://raw.githubusercontent.com/kubernetes-sigs/
            kustomize/master/hack/install_kustomize.sh" | bash
          sudo mv kustomize /usr/local/bin/
      - name: Validate Kustomize build
        run: kubectl kustomize k8s/overlays/staging

```

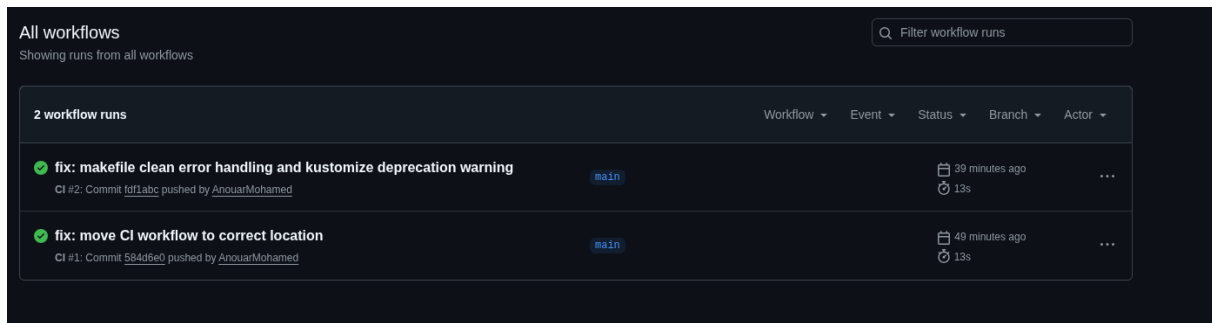


Figure 8: GitHub Actions CI run passing — lint and kustomize validation both green

8 Makefile Automation

A Makefile abstracts all cluster operations so the platform can be operated without remembering long kubectl commands:

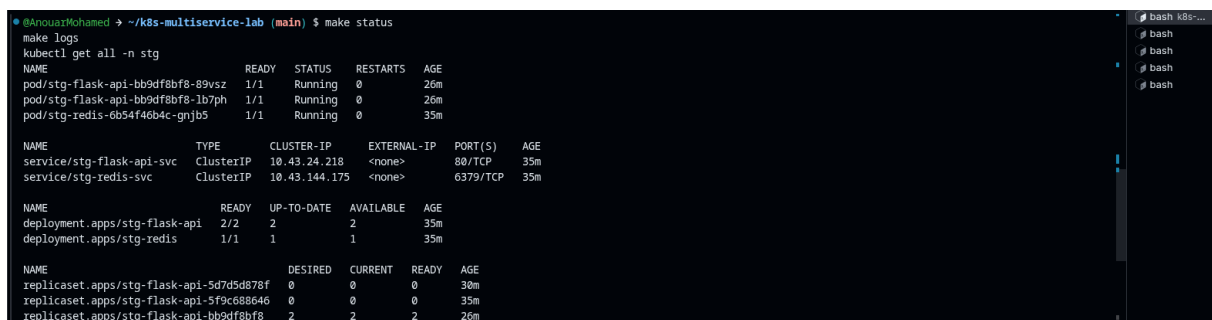


Figure 9: make status showing all resources in the stg namespace

9 Repository Structure

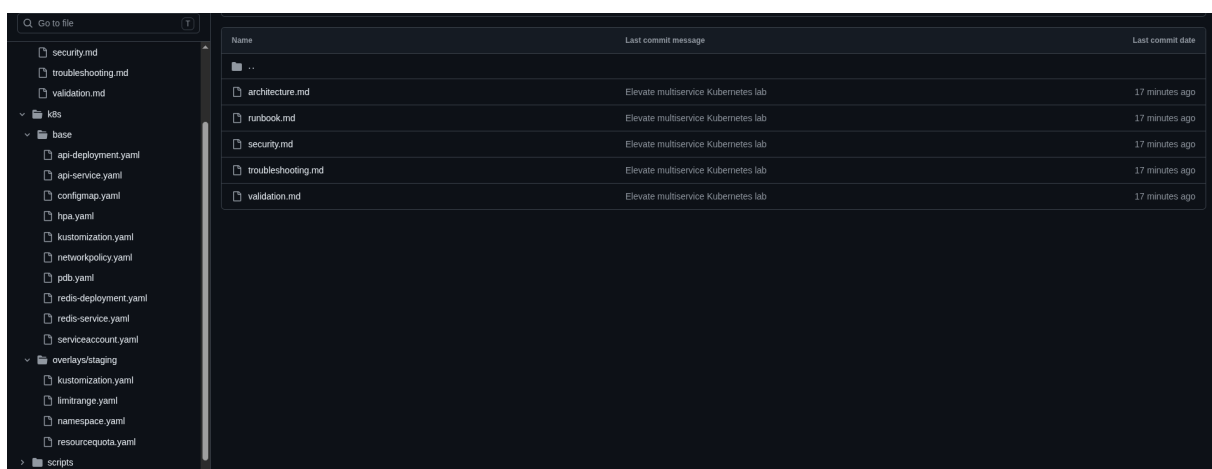


Figure 10: k8s-multiservice-lab public repository on GitHub

10 Summary

Concept	What was demonstrated
Custom image	Flask API built with Docker and loaded into k3s containerd runtime
Kustomize overlay	Base/overlay separation with namePrefix, label injection, and patches
Secrets	Imperatively created, base64 encoded, gitignored — never in version control
Inter-service DNS	Redis resolved by DNS name from inside Flask pod over Kubernetes cluster DNS
ConfigMap injection	REDIS_HOST and REDIS_PORT injected as env vars and confirmed inside container
Hit counter	Incrementing counter proves end-to-end Flask → Redis communication
CI/CD pipeline	GitHub Actions running yamllint and kustomize validation on every push
Makefile	All cluster operations abstracted into simple make targets

Table 1: *Skills demonstrated in this lab*

References

- k3s documentation: <https://docs.k3s.io>
- Kustomize documentation: <https://kubect1.docs.kubernetes.io/references/kustomize>
- Flask documentation: <https://flask.palletsprojects.com>
- GitHub Actions: <https://docs.github.com/en/actions>